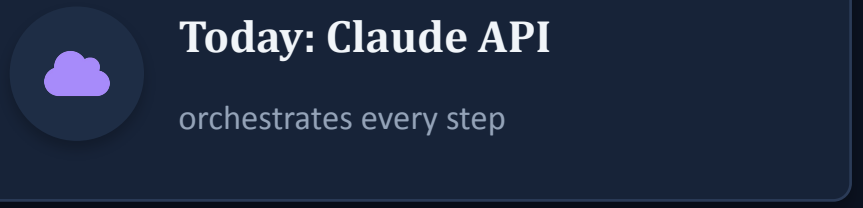
ML INFRASTRUCTURE · AGENTOS

Training an In-House Orchestrator

Replacing the Claude API at the centre of our agent harness with a self-hosted, fine-tuned orchestrator — the model that plans, routes, and decides across every turn.



Distil from Claude · reinforce against the live harness · judge with Nemotron Super

Why bring the orchestrator in-house

Sovereignty

Government workloads cannot depend on an external API for the decision core. A self-hosted orchestrator keeps planning on our own infrastructure.

Cost

Claude is invoked on every routing and planning step. At production volume that recurring spend dwarfs a one-time fine-tune.

Latency & control

A local model on bipardai02 removes network round-trips and gives us full control over the decision policy and its updates.

Scope we are committing to: fine-tune the **orchestrator** — the planner / router / decision loop of our harness — not every subagent. Leaf tools and workers stay as they are for now. We keep Claude available for prototyping only, with no government data.

What the orchestrator actually has to do

This is the reasoning seat, not a formatting layer. Every function below is a judgement call the model must learn.

Decompose

Break a task into ordered subtasks; choose sequential vs parallel.

Route

Dispatch each subtask to the right subagent, skill, or tool.

Read results back

Interpret subagent output — success, garbage, or contradiction.

Continue or stop

Decide whether the task is done or another step is needed.

Replan on failure

When a step fails, recover — don’t derail the whole trajectory.

Synthesize

Compose subagent outputs into the final answer for the user.

“Small” and “orchestrator” pull against each other

The capacity wall

A 1.5–3B model can route and format, but cannot hold a multi-step plan or recover when a subagent returns garbage. Orchestration needs reasoning headroom.

Compounding errors

Cloning expert traces only teaches states the expert visited. One wrong step lands the model in a state it never trained on — over a 15-step trajectory, errors cascade.

The reframe that resolves it

Make the workers small and keep the orchestrator capable. The orchestrator fires once or twice per turn; leaf agents fire many times — so the cheap-to-run wins come from small specialised workers, while the one component whose mistakes cascade stays strong. We do not shrink orchestration to the point of fragility; we fine-tune a capable open model and let RL close the reliability gap.

A staged build: prove the pipeline, then scale it

These are not competing options — they are two steps. Step 1 stands up a model- and dataset-agnostic fine-tuning + RL pipeline at the lowest possible cost; Step 2 reuses that exact pipeline for the real orchestrator.

STEP 1

Prove the pipeline

Smallest viable config — 4B base, QLoRA, minimal data, simple DAgger / DPO. Goal: every stage runs and connects on cheap hardware. A reusable rig for any model or dataset.

STEP 2

Scale to the orchestrator

Same pipeline, production target — capable base, GRPO + real DAgger, Nemotron judge, full data engine + eval gates, and a heavier framework when needed.

WHY STEP 1 FIRST

De-risk engineering cheaply

Pipeline bugs — data format, loss masking, reward wiring — surface on a 4B QLoRA run that finishes in minutes, not a multi-day run.

Agnostic by design

The pipeline doesn't care about model size or dataset, so what you prove at 4B transfers directly to the 26B production run.

Fast feedback loop

Small runs let us shake out the full data → SFT → RL → eval cycle many times before committing GPU-weeks.

A reusable asset

The validated rig serves every future fine-tune — not just this orchestrator — which was the original goal.

How each lever scales from Step 1 to Step 2

Same pipeline throughout. Each lever starts at its cheapest setting to validate the plumbing, then moves to the production setting for the real run.

DIMENSION	STEP 1 (validate cheaply)	STEP 2 (production orchestrator)
Target	A lightweight tool router	The orchestrator — a reasoning decision loop
Base model	Qwen / Gemma 4B	Gemma 26B-A4B / Nemotron mid · Qwen excluded
After SFT	DAgger in shadow mode	GRPO against the live harness
Recovery / shift	Disagreement mining on the expert path	Learner drives in sandbox, relabel from mistakes
Reward / labels	Trace comparison, no judge	Code checks + Nemotron Super as judge
Data & eval	Small seed set + synthetic; smoke-test metrics	Data engine + solvability filter + eval gates

Both steps keep SFT-first and QLoRA. The same data, training, and eval code runs in each — only the model, framework, and reward fidelity change.

What we fine-tune — and the size floor

RECOMMENDED

Gemma 4 26B-A4B

Recommended primary. MoE: ~4B active params = fast inference, 26B-class capability. Already in our AP Sovereign stack and clears procurement (Google-origin).

Nemotron-family mid

Strong fallback. We already designated Nemotron Super for the harness; a mid-size Nemotron is a natural, procurement-clean orchestrator base.

Two-tier split

If we want truly small: keep planning in a 14–30B model and use a 1.5–3B model only as the dispatch formatter. Easier to train, lower risk.

Procurement guardrail: Chinese-origin models stay excluded per our existing policy — so Qwen is out as a base. Gemma, Nemotron, Llama, and GPT-OSS families are all in-scope. Floor for the reasoning orchestrator is ~7–14B; below that, use the two-tier split.

Two stages, with a clear objective each

1 · SFT

Behavioural cloning of Claude’s orchestration traces. Establishes format and baseline decisions. ~80% of the lift.

2 · GRPO

Reinforcement against the live harness. Closes the reliability gap on the long tail where SFT alone derails.

DPO

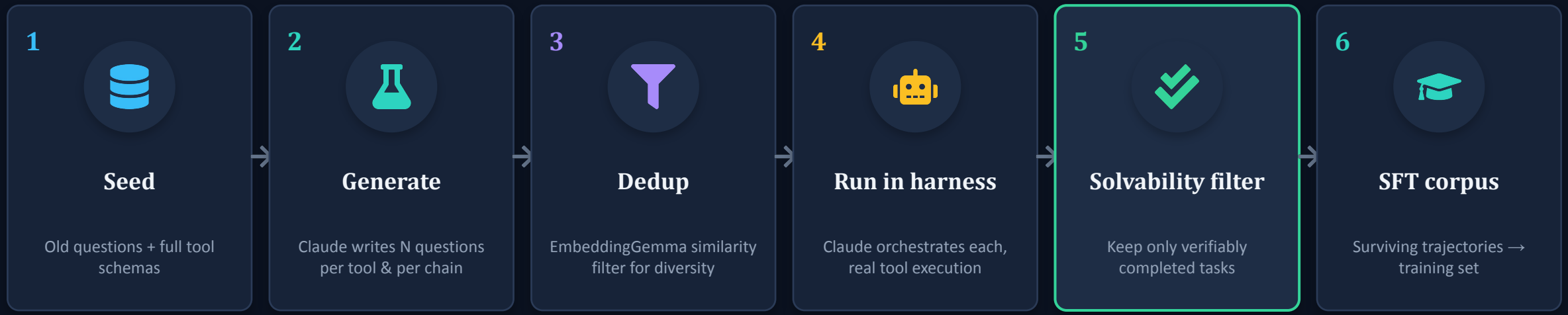
Fallback for GRPO, or to fix one systematic error.

LORA / QLORA IS A MEMORY KNOB, NOT A THIRD OBJECTIVE

Full fine-tune	All weights updated	Cleanest result; feasible for SFT of a 7–14B on 4×H100
LoRA	Small adapters, base at full precision	Good middle ground; fast iteration
QLoRA	Adapters on a quantized base	Lowest memory — needed when holding policy + reference in GRPO

How we turn old questions + the toolset into training data

One pass produces a filtered, solvable question bank AND the SFT trajectories — because generation and validation are the same step.



Never hallucinate observations

The harness must execute real tools — fake tool outputs train the model to expect a fictional world and it derails on the first real call.

Unsolvable questions are poison

A generated question the toolset can't satisfy yields a bad trajectory. The solvability filter quarantines failures for inspection, not silent deletion.

What an orchestrator training record contains

RECORD SCHEMA (train on assistant turns only)

```
{
  state: { task, plan, history },
  available: [ subagents, skills, tools ],
  target: {
    reasoning: {
      decompose, route_choice,
      continue_or_stop
    },
    action: { dispatch | finish }
  }
}
```

Keep the reasoning in the trace and train on it — for an orchestrator, decision quality depends on it. Mask loss on injected observations.

COVERAGE THAT MAKES OR BREAKS IT

- **Failure & recovery**
  Subagent errors, empty or contradictory results — the highest-value trajectories.
- **Decomposition variety**
  Not only clean 3-step plans; varied breadth and depth.
- **Every tool exercised**
  Including rare ones; per-tool generation guarantees this.
- **‘No action needed’ cases**
  When to stop or answer directly — a common small-model failure.
- **Parallel dispatch**
  If the harness supports concurrent subagents.

Reinforcement: the dataset is trivial, the reward is the work

Sample G completions per orchestration state (G ≈ 8–16), score each, optimise toward the relatively better ones. Dataset = prompts only.

Verify in code (free, deterministic)

- Schema validity — dispatch parses & matches signature
- Correct subagent / tool selected for the intent
- Execution success — sandbox runs without throwing
- Format & clean termination — no runaway loops
- Efficiency — fewer steps / sub-calls for same outcome

Judge with Nemotron (unverifiable only)

- Was the decomposition sensible & complete?
- Was this the right subagent for the subtask?
- Was stopping here correct vs continuing?
- Terminal task success vs ground truth

Sparse terminal reward → densify with process rewards on intermediate steps, or RL is glacial.

What the RL stages actually consume

Unlike SFT, neither RL record stores a target action. GRPO scores live completions; DPO compares two stored ones.

GRPO — prompts only

```
{
  prompt: [ messages up to the
            decision point ],
  meta: {
    task_id, available_tools,
    ground_truth   // for reward
  }
}
```

Completions are sampled live (G per prompt); reward = code checks + judge + terminal success. No action is stored.

DPO — preference pairs

```
{
  prompt:    [ ... ],
  chosen:    { reasoning, action },
  rejected:  { reasoning, action }
}
```

chosen = Claude's action or the top-scored sample. **rejected** = a pre-RL model mistake or the lowest-scored sample. Built offline — no live rollouts.

Reuse, don't rebuild: every RL prompt is just the state half of an SFT record, and DPO pairs fall out of the same harness rollouts used for GRPO scoring. One trajectory store feeds all three stages — SFT targets, GRPO prompts + rewards, and DPO chosen/rejected pairs.

Nemotron Super as the self-hosted RL judge

Why self-host the judge: GRPO runs G rollouts × many steps × many prompts — judge volume is enormous. On the Claude API that's a budget bonfire; on Nemotron Super (bipardai02, already deployed via vLLM) it's compute we own. Bonus: it decorrelates from Claude, which wrote the SFT data.

Never judge the verifiable

If a check can be code, make it code. Letting the judge score what a parser could adds cost and noise for nothing.

Rubric, not vibes

Give discrete scores per named dimension, not 'rate 1–10'. Vague judges get reward-hacked within a few hundred steps.

Pairwise > absolute

Score which of the G completions routed better. Relative scoring is more stable and matches what GRPO normalises anyway.

Train on the model’s own mistake states

The single biggest cause of agent failure: the model errs at step 3, lands in a state no expert trace ever visited, and has no idea what to do.

Shadow mode is not enough

If the teacher always executes the real action, the trajectory only ever follows the expert path. The model’s wrong step never propagates, so you never see the derailed state. That is disagreement mining — not distribution-shift correction.

Let the learner drive — in the sandbox

Periodically execute the model’s own action so the trajectory enters states it actually reaches, then have Claude re-label the right move FROM that derailed state. Safe because rollouts run in our sandboxed execution layer, never production.

THE AGGREGATION LOOP

1. **Model drives in sandbox** →
2. **Hits a derailed state** →
3. **Claude re-labels best action** →
4. **Add to dataset** →
5. **Retrain**

The golden set — our single source of truth

A frozen, human-verified, held-out set that is never trained or tuned on. Every promotion decision is read off it.

ONE GOLDEN RECORD

```
{
  task, available_tools,
  gold: {
    route_sequence,
    final_answer,
    max_steps
  },
  tags: [ "recovery",
          "ambiguous", ... ]
}
```

WHAT IT MUST CONTAIN

- Held-out real tasks — same distribution as training, excluded from it
- Hard cases over-represented: recovery, ambiguous routing, stop/continue
- Ground truth per item: expected route, final outcome, ideal step count
- Every tool covered, stratified so you can read per-tool subscores

HYGIENE RULES

- Freeze & version it — zero leakage into any training stage
- Human-verified labels for ambiguous items, not just Claude’s output
- A few hundred well-chosen trajectories beats a noisy thousand

The data engine generates the training corpus; the golden set is curated and frozen separately so the two never mix.

Define success before we spend GPU time

Every promotion decision — keep training, swap the base, ship — is gated on these. Without them, ‘is it good enough?’ has no answer.

Routing accuracy

Does the model dispatch to the same subagent Claude did?

Argument quality

Are dispatch arguments correct, not just the target name?

End-task completion

Does the full trajectory finish the task — the metric that matters most.

Recovery rate

After a wrong step, does it get back on track? Tests the DAgger loop directly.

Stop precision

Terminates at the right moment vs looping or quitting early.

Cost & latency delta

Measured savings vs the Claude baseline — the business case, quantified.

Everything runs on infrastructure we already own

Compute

bipardai02 — 4× H100 NVL. Full SFT of a 7–14B fits; QLoRA for GRPO holds policy + reference in memory.

Serving

vLLM for fast rollouts and the Nemotron judge endpoint; both already running on the box.

Tooling

Unsloth for LoRA/QLoRA SFT + lightweight GRPO; verl or OpenRLHF if we scale GRPO across GPUs.

COMPONENT MAP

Teacher (SFT)	Claude API — traces only, no gov data
Base / policy	Gemma 4 26B-A4B (primary) · Nemotron mid (fallback)
RL judge	Nemotron Super 120B — self-hosted via vLLM
Dedup embeddings	EmbeddingGemma 300M (768-dim)
Execution	Sandboxed execution layer — real tools, safe rollouts

Phased delivery with a decision gate each stage

PHASE 1

Pipeline rig

Step 1 — build the model/dataset-agnostic SFT+RL+eval loop; validate on 4B + QLoRA + a toy task.

Gate: end-to-end run, metrics move

PHASE 2

Data + SFT

Step 2 begins — build the data engine, fine-tune the capable base on real traces.

Gate: format + routing accuracy

PHASE 3

GRPO + DAgger

Reinforce against the harness; add sandbox rollouts & re-labelling for recovery.

Gate: recovery + completion lift

PHASE 4

Shadow & cutover

Run beside Claude on live traffic, then cut over with rollback.

Gate: parity at lower cost

DECISIONS TO LOCK

Three calls to make before Phase 1

01

Reasoning depth of the orchestrator

How much planning does the model own vs the rest of the harness? This sets whether we need full reasoning traces and a 7–14B+ base, or a leaner dispatch model.

02

Base model & size

Gemma 4 26B-A4B as primary, or commit to a Nemotron mid? Confirm against latency and capability targets.

03

Procurement scope

Confirm this orchestrator inherits the no-Chinese-models rule (it should) — which fixes the candidate list to Gemma / Nemotron / Llama / GPT-OSS.

The path is clear. Distil from Claude, reinforce against our own harness, judge with Nemotron — all on hardware we run. Lock these three calls and Phase 1 can start immediately.